How to use the handheld camera

Erika Brattich

Department of Physics and Astronomy "Augusto Righi"

Settings

Before taking the photo (mandatory, unchangeable after):

- Adjust the focus of the camera
- Set the range of temperature (set by us)
- Determine the area of interest

Other settings adjustable after the image is acquired:

- Level and span
- Color table
- Object (Ambient) parameters

Focus

An image with an incorrect focus is useless!

To adjust the focus, point the camera on a surface with square geometry and large thermal contrast

A wooden table on a marble floor

Area of interest

- It defines the area we want to capture with the shot
- The subject of our investigation should be in the centre of the image
- The cameraman should be at a convenient distance from the object and a convenient angle to avoid self reflection

Level and span

- The span is the range of the thermal scale (not the temperature range) on the screen (it is defined by the minimum and maximum temperature the camera is reading at each moment; it is based on what we look at).

- The level is the central value of the span (located in the photographic area).

Color table

•It defines how we are going to visualize the thermal image (remember the raw thermal image is in black and white)

•It helps detailing certain aspects depending on the choice of color table

•Rule of Thumb: use high-contrast color table for images with low span, low-contrast color table for images with high span

Ambient parameters

- Parameters to be set to make our measurement realistic
- To change the parameters means to change the temperature

Setup the ambient parameters

- The ambient parameters are emissivity, reflective apparent temperature, air temperature, relative humidity and distance (plus transmittance compensation when a transmitting body is considered).

- Setting these parameters allow to compensate for the object radiative properties and the filter of the atmosphere

Operate in "apparent temperature" mode

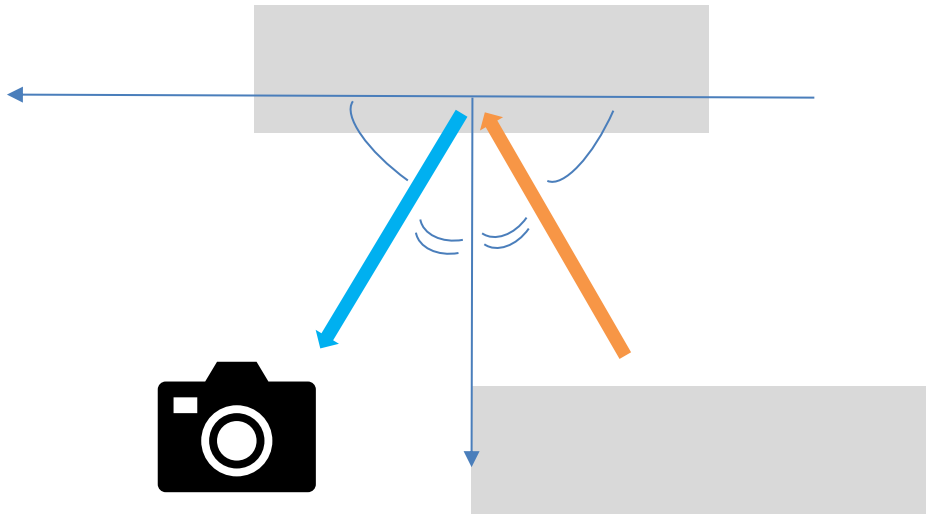
- The apparent temperature is the temperature measured by the infrared camera if we assume that all the radiation comes from a black body near the camera

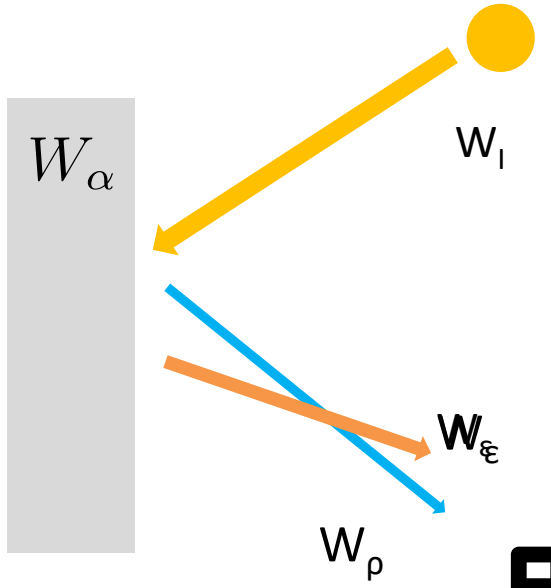
- To setup the camera for the apparent temperature mode, you must set the emissivity to 1 and the distance to 0 m

- The more the emissivity of a body approaches 1, the more the apparent temperature is the real temperature of the object

Reflective apparent temperature – an estimation of the ambient temperature

- It is the apparent temperature of the objects which are reflected to the infrared camera by the object we want to analyze
- It is not the atmospheric temperature!!

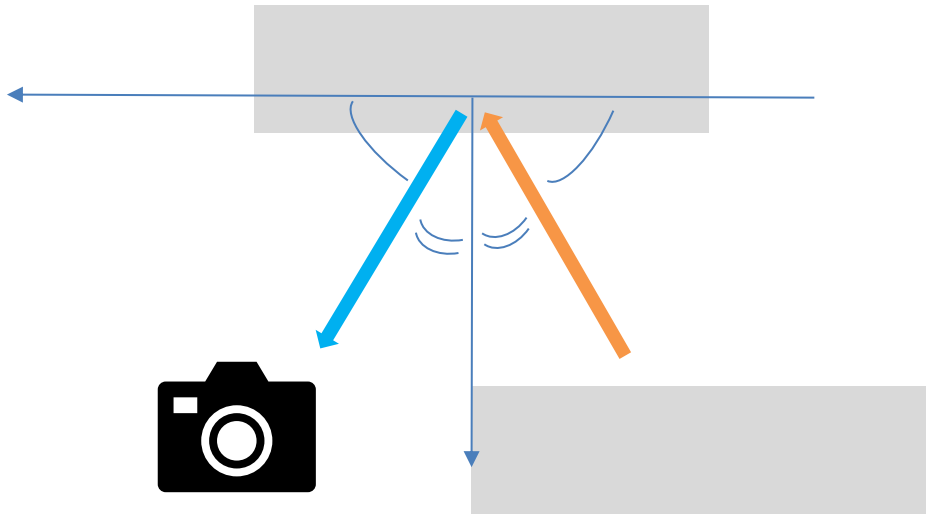




Reflective apparent temperature – an estimation of the ambient temperature

- How to measure the reflective apparent temperature

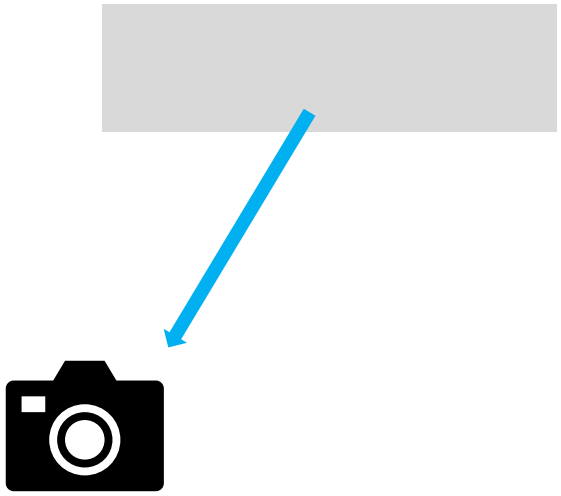
- Direct method: In apparent temperature mode, I select the angle between myself and the object and then search the main reflective source at 180° - the angle



Reflective apparent temperature – an estimation of the ambient temperature

- How to measure the reflective apparent temperature

- Official method: Apply an aluminum foil (high reflective matter) on the object and measure only the value on it



Reflective apparent temperature – an estimation of the ambient temperature

•Given the object emissivity and apparent reflective temperature, the infrared camera automatically computes the real reflectance

$$\rho = 1 - \epsilon$$

- and removes the reflective radiation

$$W_{refl} = \rho \sigma T_{refl}^4$$

Emissivity

- Measure the emissivity of the object you are going to measure before taking the snapshot
- Tabled emissivity for different materials are good approximation but do not consider impurities, dirtiness and roughness
- The best option would be to produce our own emissivity table

High vs low emissivity

In bodies with high emissivity (non-metal objects):

- Apparent and real body temperature are close
- The apparent temperature is already a good measurement

In bodies with low emissivity (non-oxidized metal objects):

- The apparent temperature is close to the ambient temperature
- The camera will see more reflected then emitted radiation
- Temperature estimation will not be reliable even compensating for the emissivity

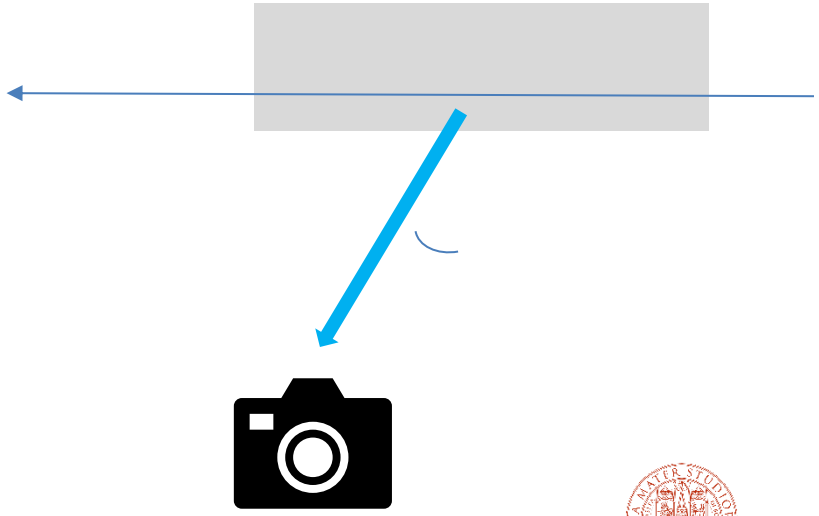
Factors that modify the emissivity

Material: trivial

Surface: a roughness surface has an apparent larger emissivity than a smooth one

Geometry: cavities increases emissivity

Angle: you should always put yourself at an angle larger than 0° (to avoid self-reflection) and less than 50° to take the shot. A 10-15° angle is optimal



Factors that modify the emissivity

Wavelength: the emissivity on my camera can be different from another if the range of measured wavelength is different

Temperature: high/low temperature may modify the state of the material

Impurities: dirty, oxidation, irregularities modify the emissivity

Measuring conditions

The object must have:

- Zero/Negligible transmittance (opaque)
- A known emissivity
- A known reflectivity (computed from emissivity)
- A location on an ambient context we can evaluate and exclude

Experience 4 – Thermal camera

Dr. Erika Brattich

Department of Physics and Astronomy "Augusto Righi",
University of Bologna

The FLIR thermal camera

Risoluzione IR	1024 × 768 pixel; UltraMax: 3,1 MP
Sensibilità termica/NETD	<20 mK a +30 °C
Campo visivo (FOV)	12° × 9°
Fotocamera digitale	Campo visivo si adatta all'ottica infrarosso
Modalità immagine	Termica, termica MSX, picture in picture, fotocamera digitale
Display	Touchscreen LCD wide screen integrato da 4,3", 800 × 480 pixel
Supporto di memorizzazione	Scheda di memoria SD o SDHC. Consigliato Classe 10 o superiore.

Altre specifiche: [FLIR T1020 | Teledyne FLIR]

Infrared experiments of thermal energy and heat transfer

You will explore thermal energy, thermal equilibrium, heat transfer (convection, diffusion) in one hands-on activity augmented by the thermal vision through an infrared (IR) camera.
Each activity will guide you to learn basic science concepts by doing experiments following the Predict-Observe-Explain inquiry process illustrated in Figure 1. You will set up an experiment, predict what will happen, conduct the experiment, observe what happens using an IR camera, and explain the observed results. In the case where your prediction does not agree with your observation, you can redo the experiment to double-check the results. If the results are still not what you expected, you should resolve the conflict: Is your result or your prediction wrong? At the end of each activity, you will apply what you have learned to answer some questions that help generalize your understanding to other situations.

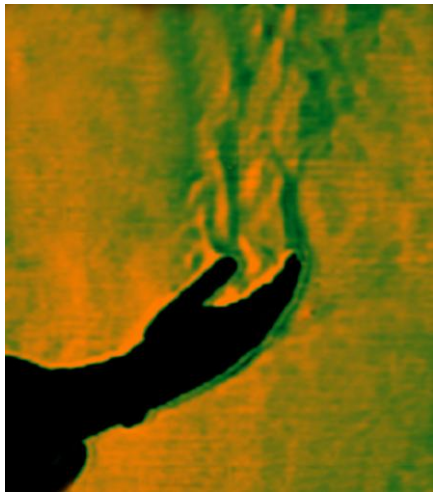
Figure 1

Scope

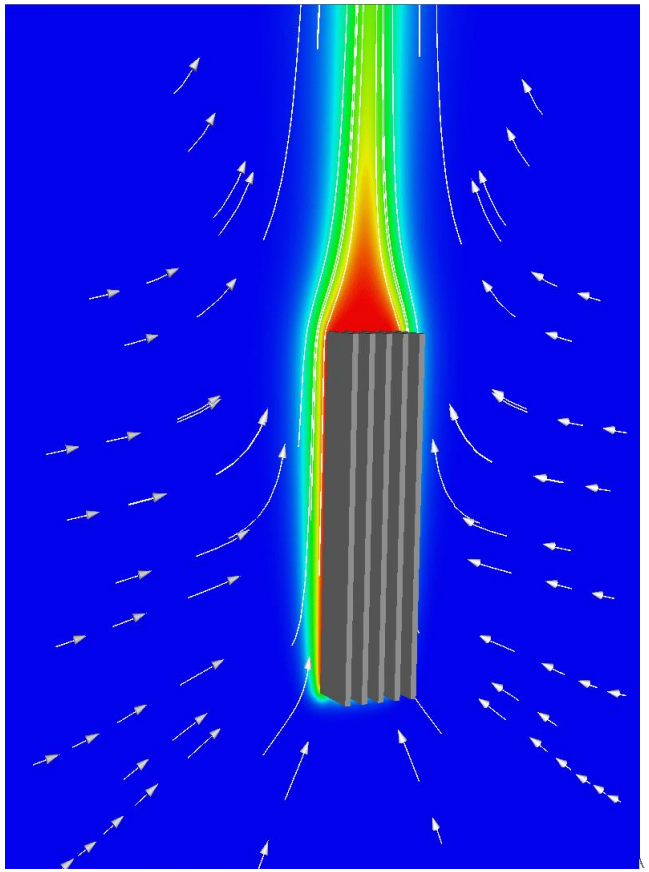
- Observe the effects of natural convection, diffusion and forced convection, thermal conduction and thermal radiation

Part 1: natural convection

Natural convection is the flow of thermal energy along with fluid motion that occurs without any external force. Any temperature difference in a fluid on the Earth will cause natural convection and result in redistribution of thermal energy.



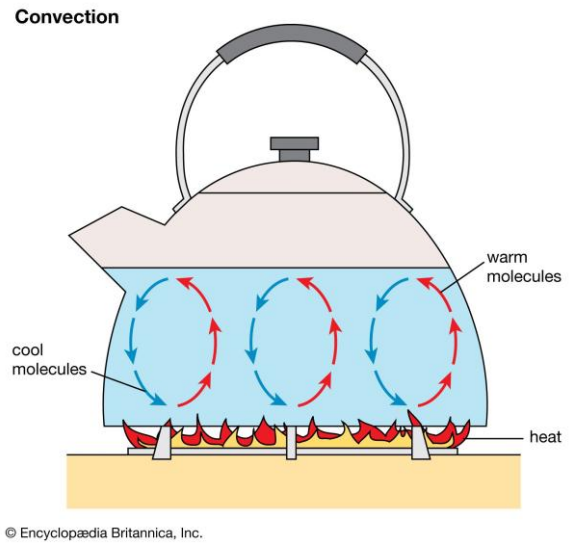
thermal convection originating from heat conduction from a human hand (in silhouette) to the surrounding still atmosphere, initially by diffusion from the hand to the surrounding air, and subsequently also as advection as the heat causes the air to start to move upwards.



How does convection works?

Convection works by **areas of a liquid or gas heating or cooling greater than their surroundings**, causing differences in temperature. These temperature differences then cause the areas to move as the hotter, less dense areas rise, and the cooler, more dense areas sink.

Often the areas of heating and cooling are fixed, and allow convective cycles or currents to become established. For example, a saucepan of water over a flame may develop convective currents as the water is heated from below, rises to the surface, and cools. Once cooled enough, the water then sinks back to the bottom of the saucepan where the cycle is repeated, and the convective overturning continues.

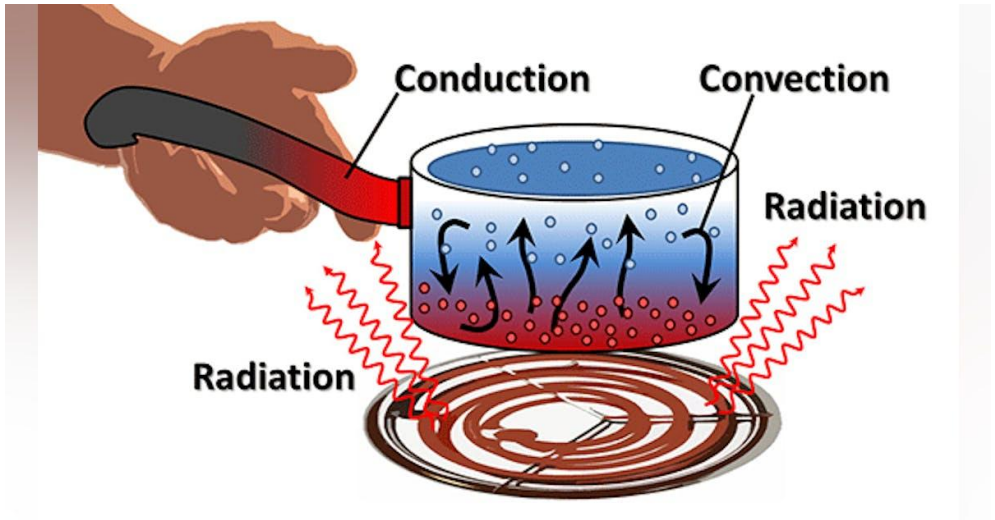


Why does convection occur?

Convection is a vital process which helps to **redistribute energy away from hotter areas to cooler areas of the Earth**, aiding temperature circulation and reducing sharp temperature differences. Without convection simple tasks such as boiling water in a kettle would be much slower as only the water directly in contact with the heat ring at the bottom of the kettle would be able to be heated, with the water at the top staying cool.

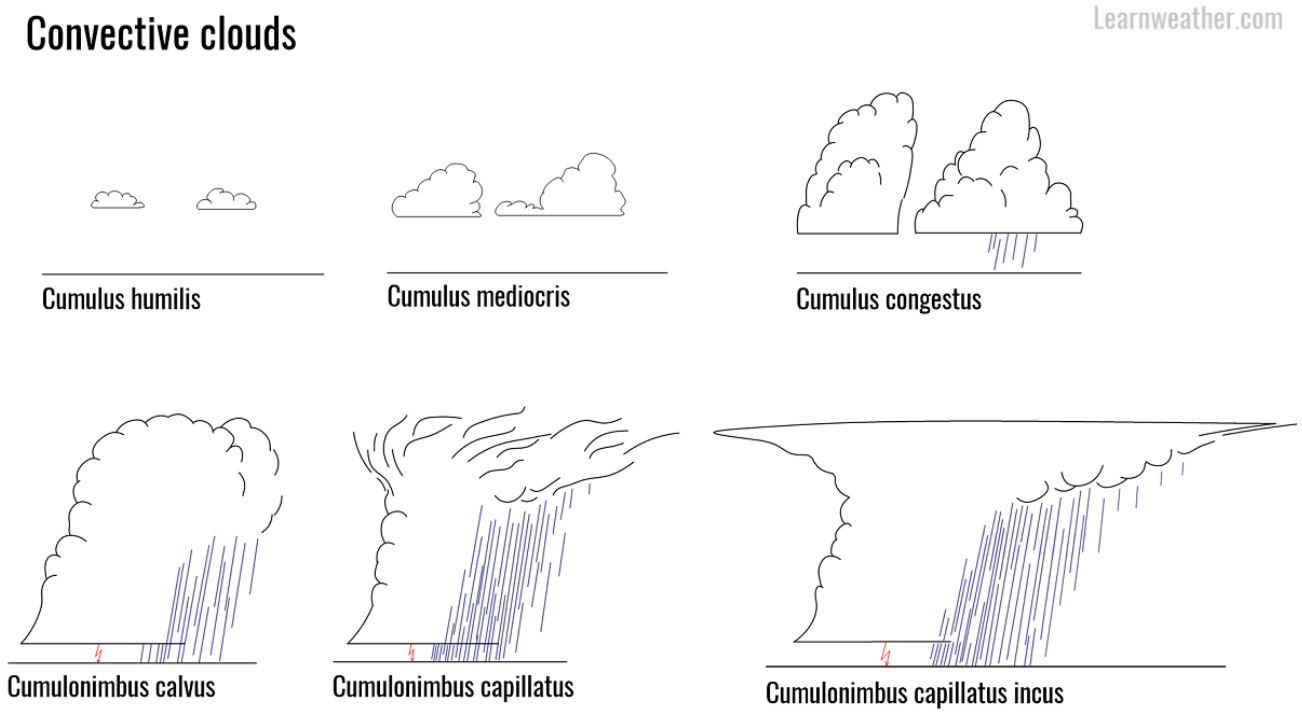
How is convection different from conduction?

Convection is the movement of particles through a substance, transporting their heat energy from hotter areas to cooler areas. **Conduction** however, doesn't necessarily involve particles moving. Instead energy is passed from one particle to another **upon contact**, transferring heat. As a result, conduction in liquids and gases is a much slower process than convection, as particles are free to move and direct contact is reduced. However, conduction is much more effective in solids than convection, as the particles are densely packed, continuously touching one another to allow an efficient transfer of heat. Additionally, in solids particles fixed and unable to move, stopping the transfer of energy via convection.



How does convection affect the weather?

Convection within the atmosphere can often be observed in our weather. For example, as the sun heats the Earth's surface, the air above it heats up and rises. If conditions allow, this air can continue to rise, cooling as it does so, forming Cumulus clouds. Stronger convection can result in much larger clouds developing as the air rises higher before it is cooled, sometimes producing Cumulonimbus clouds and even thunderstorms.

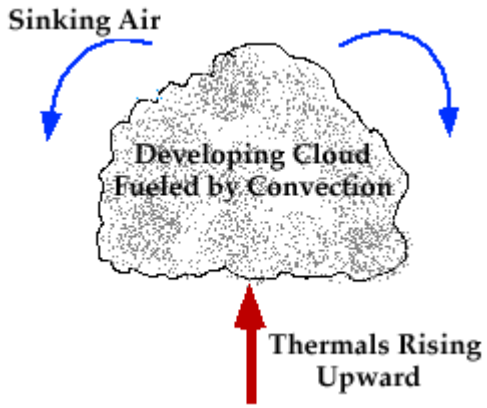


Can convection be seen?

Convection is the rising motion of warmer areas of a liquid or gas, and the sinking motion of cooler areas of liquid or gas, sometimes forming a complete cycle. However, it is often difficult to see, particularly within the air. Nevertheless, if convective clouds form during a sunny day, they can often be observed growing in size and getting taller as more and more air rises from the surface and condenses higher up into the cloud.

Convection in meteorology

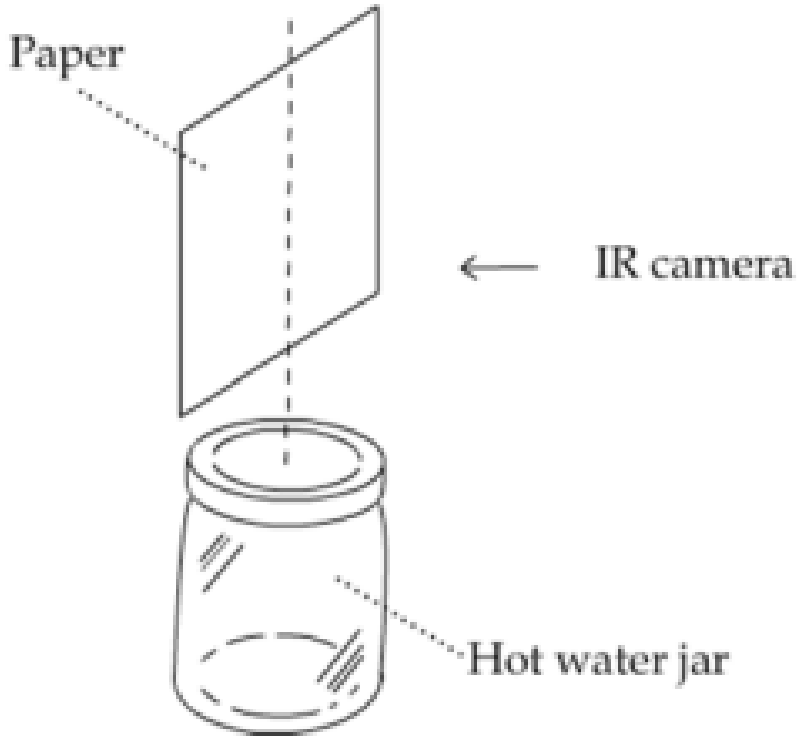
In meteorology, convection refers primarily to atmospheric motions in the vertical direction. As the earth is heated by the sun, different surfaces absorb different amounts of energy and convection may occur where the surface heats up very rapidly. As the surface warms, it heats the overlying air, which gradually becomes less dense than the surrounding air and begins to rise



The bubble of relatively warm air that rises upward from the surface is called a "thermal".

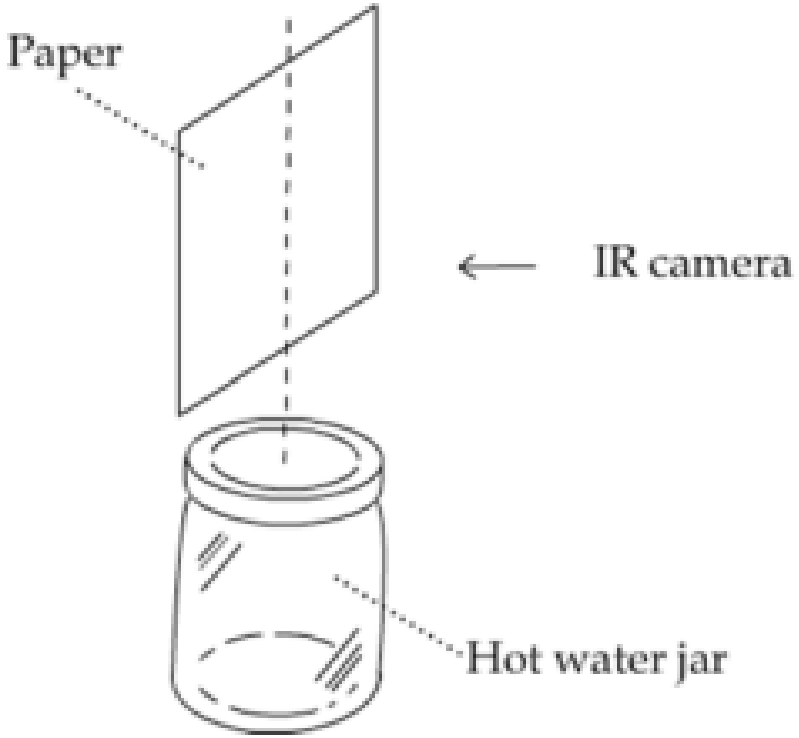
Material and tools

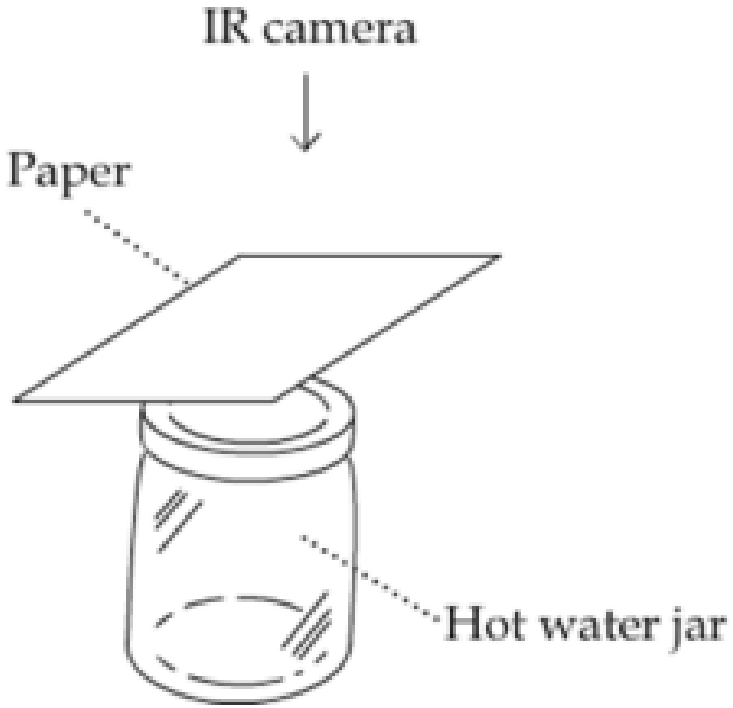
- An IR camera
- A black paper
- A cup of hot water (>80°C)



Prediction

What will happen to the temperature pattern of a piece of paper when you hang it above a hot water cup in the vertical and horizontal directions?





Observation

Follow these steps to conduct the experiment and observe the convective heat flow that naturally occurs in the air through the IR camera:

1. Put a jar of hot water on a table where there is no other hot or cold objects.
2. Piece of dry black paper above the jar in the vertical direction. Make sure that the paper is close to the top of the cup but does not touch it.
3. Observe the paper through the IR camera. Make sure that the hot water jar does not show up in the IR view. We will take an IR image every 1 second (1Hz).
4. Paper in horizontal position above the mug. Repeat the observation.

Explanation

Use the IR images that your group has taken as evidence to check your prediction.
Whether your prediction is correct or not, explain your observation based on the concepts of **natural convection, thermal conduction (between air and the jar and between air and the paper), and thermal radiation (between the paper and the jar)**. Back your explanation with your IR images.

1. What happened to the temperature pattern of a piece of paper when you hung it above a hot water jar in the vertical direction?
2. What happened to the temperature pattern of a piece of paper when you hung it above a hot water jar in the horizontal direction?

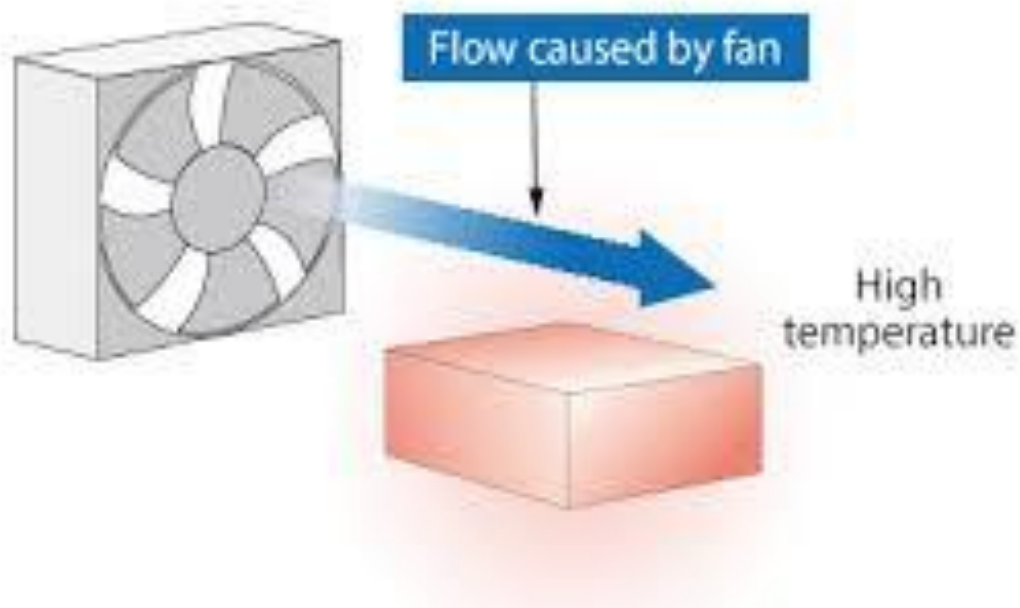
Questions

Based on what you have learned from this activity, answer the following questions:
1. How can you distinguish the effects of heat transfer on a piece of paper from a hot water jar through radiation and convection? Suppose the paper faces the hot water jar from the above position.
2. Which part of a heated room tends to be warmer, near the ceiling or near the floor? Explain your answer.

For each part of the experiment, choose the best representative images.
Explain what you have observed using the images and temperature statistics associated.
Is diffusion occurring homogeneously?

Part 2: forced convection

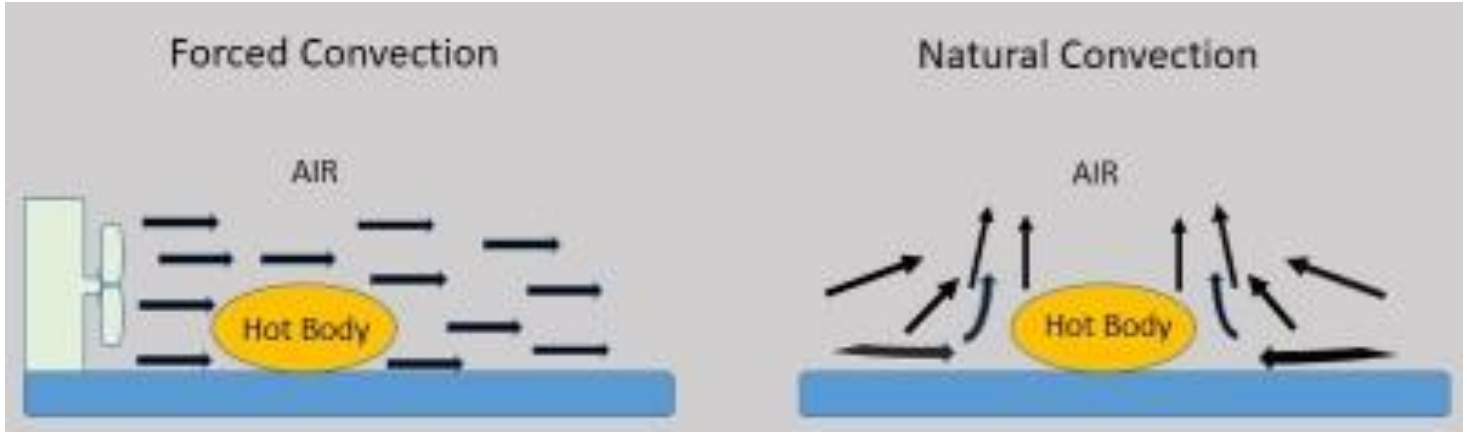
Unlike natural convection, forced convection is the flow of thermal energy along with fluid motion driven by an external force such as wind.



Forced convection

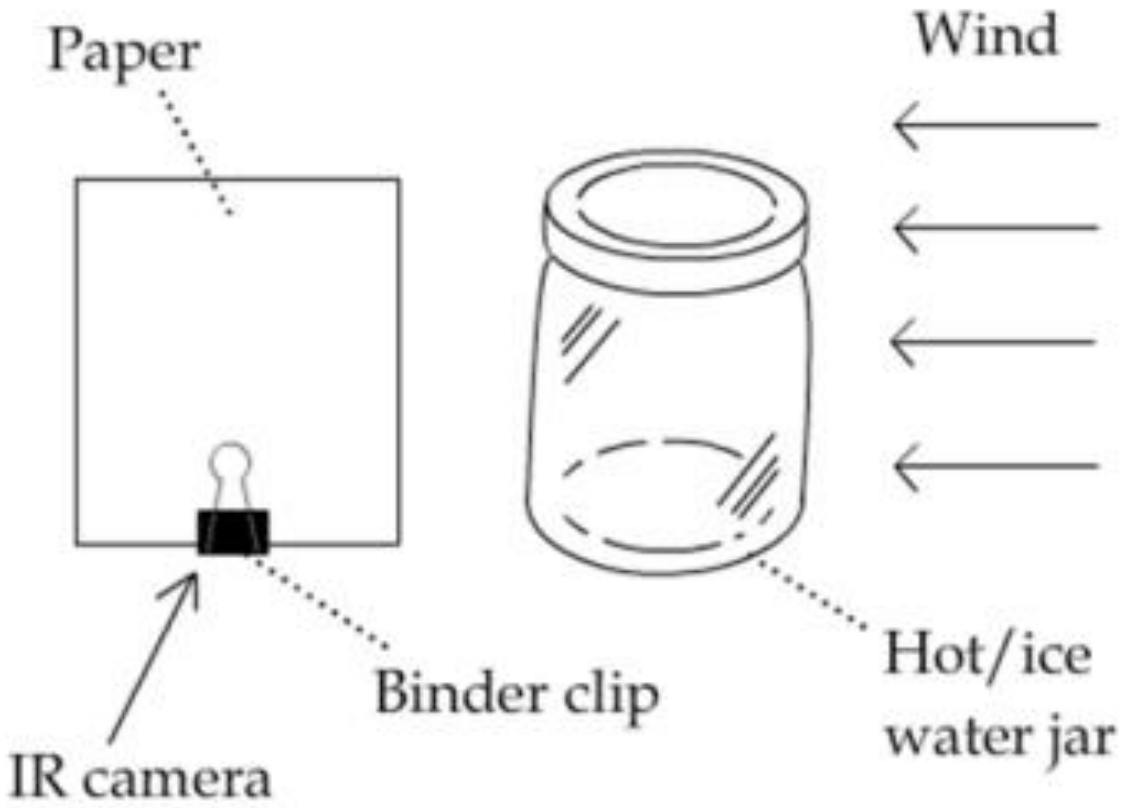
This type of convection occurs when a fluid is forced to flow over a surface or through a conduit by an external means, such as a fan, pump, or wind. In the case of wind, it acts as an external force that drives the movement of air, causing forced convection. Wind is a common example of forced convection in the atmosphere, where the movement of air is driven by pressure differences and temperature gradients.

In the context of wind, the convection caused by wind would be considered forced convection because the movement of air is driven by external forces rather than solely by natural buoyancy effects.



Material and tools

- An IR camera
- A black paper
- A cup of hot water
- A hand-held fan



Prediction

Before you do anything, answer the following questions:

1. What will happen to the temperature pattern of a piece of paper next to a hot water jar when wind is blowing towards them?

Observation

Follow these steps to conduct the experiment and observe forced convection caused by blowing air through the IR camera:

1. Put a cup of hot water on a table where there is no other hot or cold objects.
2. Place the paper about 1" away from the jar. Make sure that the paper does not touch or face the jar
3. Keep the paper in that position for at least 30 seconds while you use the hand-held fan to blow air towards the jar. Observe the paper through the IR camera. Make sure that the hot water jar does not show up in the IR view. IR images will be taken every 1 second.

Explanation

Use the IR images that you and your partner have taken as evidence to check your prediction. Whether your prediction is correct or not, explain your observation based on the concepts of **forced convection, fluid flow, and thermal conduction** (between air and the paper and between air and the jar). Back your explanation with your IR images.

1. What happened to the temperature pattern of a piece of paper next to a hot water jar when wind was blowing towards them?

Questions

Based on what you have learned from this activity, answer the following questions:
1. What can you do to increase the heat transfer from the hot water jar to the paper through forced convection? List all possible methods and explain them.
2. Why do people turn on the ceiling fan in a heated room in the winter when there is no need to cool? Explain your answer.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Report

Instructions

- Maximum 7 pages
- Deadline 2 weeks from the experiment, but please allow enough time for evaluation if you aim to do examination on 10th June
  - 12 June for exams after 10 June
  - **6 June** for exams on 10 June
- Provide images or other supporting materials (e.g., tables with quantities, plots) whenever necessary to support your statements.

For the report, you will receive all the images (1 frame x 1 Hz) collected during the experience in jpg (or any other format you prefer), along with a report statistics for the central position of the image and other positions (cursor 3x3, box, … explanations in the lab). We will divide the experience in the two setups, horizontal and vertical paper (cup always below it).

Eventually, csv matrix files can also be exported. Given the objectives of the experience, it will suffice to observe carefully the processes, take notes during it (e.g., times, frame, details, pictures with smartphone), comment some selected images aside some statistics to discuss and explain what you have observed during the different (4) phases of the experiment.

You have also two short (R or Python) scripts to extract easily and plot time series statistics of thermal images.

Eventually the following slides offer R and Python libraries to work with thermal images.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Work with thermal images

R libraries to analyse acquired images

Work with thermal images / videos

In R, two dedicated libraries:

1) Thermimage (https://github.com/gtatters/Thermimage)
2) ThermStats (https://github.com/rasenior/ThermStats)

Thermimage

- Functions for importing FLIR image and video files (limited) into R.
- Functions for converting thermal image data from FLIR based files, incorporating calibration information stored within each radiometric image file.
- Functions for exporting calibrated thermal image data for analysis in open source platforms, such as ImageJ.
- Functions for running command line conversion tools to prepare FLIR thermal image files for direct import into ImageJ.
- Functions for steady state estimates of heat exchange from surface temperatures estimated by thermal imaging.
- Functions for modelling heat exchange under various convective, short-wave, and long-wave radiative heat flux, useful in thermal ecology studies.

Thermimage

Install

install.packages("Thermimage", repos='http://cran.us.r-project.org')

Requirements

Developed on OSX, works well on Linux. Many features in Windows will require installation of command line tools that may or may not work as effectively. The internal R functions should operate fine in Windows.

Exiftool is required for certain functions. Installation instructions can be found here: http://www.sno.phy.queensu.ca/~phil/exiftool/install.html (Windows users might find his instructions difficult to follow, so perhaps try this link: https://oliverbetz.de/pages/Artikel/ExifTool-for-Windows). I prefer the simple solution of installing exiftool.exe directly into the windows folder and then verifying it has security privileges to run.

Imagemagick is required for certain functions. Installation instructions can be found here: https://www.imagemagick.org/script/download.php

Perl is required for certain functions. Installation instructions can be found here: https://www.perl.org/get.html

To load a FLIR JPG, you first must install Exiftool as per instructions above. Open sample flir jpg included with Thermimage package:

```
library(Thermimage)
f<-paste0(system.file("extdata/IR_2412.jpg", package="Thermimage"))    #modify as appropriate #directory and filename
img<-readflirJPG(f, exiftoolpath="installed")
dim(img)
```

The readflirJPG function has used Exiftool to figure out the resolution and properties of the image file.

Extract meta-tags from thermal image file

cams<-flirsettings(f, exiftoolpath="installed", camvals="")
head(cbind(cams$Info), 20) # Large amount of Info, show just the first 20 tages for readme

```
##                     [,1]
## ExifToolVersionNumber 12.26
## FileName              2412
## Directory             ".4.0"
## FileSize              638
## FilePermissions       "--"
## FileType              ""
## FileTypeExtension     ""
## MIMEType              ""
## JFIFVersion           1.01
## ExifByteOrder         "-"
## Make                  ""
## CameraModelName       660
## Orientation           ""
## XResolution           72
## YResolution           72
## ResolutionUnit        ""
## Software              "1.1.98"
## YCbCrPositioning      ""
## ExposureTime          133
## ExifVersion           220
```

This produces a rather long list of meta-tags. If you only want to see your camera calibration constants, type:

*plancks<-flirsettings(f, exiftoolpath="installed", camvals="-*Planck*")*

unlist(plancks$Info)

```
##     PlanckR1      PlanckB      PlanckF      PlanckO      PlanckR2
##  2.110677e+04  1.501000e+03  1.000000e+00 -7.340000e+03  1.254526e-02
```

If you want to check the file data information, type:

cbind(unlist(cams$Dates))

```
##                           [,1]
## FileModificationDateTime "2021-09-23 18:46:07"
## FileAccessDateTime       "2021-09-23 19:08:42"
## FileInodeChangeDateTime  "2021-09-23 18:46:09"
## ModifyDate               "2013-05-09 16:22:23"
## CreateDate               "2013-05-09 16:22:23"
## DateTimeOriginal         "2013-05-09 22:22:23"
```

or just:

cams$Dates$DateTimeOriginal

Convert raw binary to temperature

Now you have the img loaded, look at the values:

str(img)

int [1:480, 1:640] 18090 18074 18064 18061 18081 18057 18092 18079 18071 18071 ...

If stored with a TIFF header, the data load in as a pre-allocated matrix of the same dimensions of the thermal image, but the values are integers values, in this case ~18000. The data are stored as in binary/raw format at 2^16 bits of resolution = 65536 possible values, starting at 0. These are not temperature values. They are, in fact, **radiance values or absorbed infrared energy values** in arbitrary units. That is what the calibration constants are for. The conversion to temperature is a complicated algorithm, incorporating Planck's law and the Stephan Boltzmann relationship, as well as atmospheric absorption, camera IR absorption, emissivity and distance to name a few. Each of these raw/binary values can be converted to temperature, using the *raw2temp* function:

temperature<-raw2temp(img, ObjectEmissivity, OD, ReflT, AtmosT, IRWinT, IRWinTran, RH,
PlanckR1, PlanckB, PlanckF, PlanckO, PlanckR2, ATA1, ATA2, ATB1, ATB2, ATX)

str(temperature)

num [1:480, 1:640] 23.7 23.6 23.6 23.6 23.7 …

The raw binary values are now expressed as temperature in degrees Celsius. Let's plot the temperature data:

library(fields) # should be imported when installing Thermimage
plotTherm(temperature, h=h, w=w, minrangeset=21, maxrangeset=32)
#modify temperature range as appropriate

The FLIR jpg imports as a matrix, but default plotting parameters leads to it being rotated 270 degrees (counter clockwise) from normal perspective, so you should either rotate the matrix data before plotting, or include the *rotate270.matrix* transformation in the call to the *plotTherm* function:

plotTherm(temperature, w=w, h=h, minrangeset = 21, maxrangeset = 32, trans="rotate270.matrix")
#care of temperature range!

If you prefer a different palette:

plotTherm(temperature, w=w, h=h, minrangeset = 21, maxrangeset = 32, trans="rotate270.matrix", thermal.palette=rainbowpal)

plotTherm(temperature, w=w, h=h, minrangeset = 21, maxrangeset = 32, trans="rotate270.matrix", thermal.palette=glowbowpal)

plotTherm(temperature, w=w, h=h, minrangeset = 21, maxrangeset = 32, trans="rotate270.matrix", thermal.palette=midgreypal)

plotTherm(temperature, w=w, h=h, minrangeset = 21, maxrangeset = 32, trans="rotate270.matrix", thermal.palette=midgreenpal)

plotTherm(temperature, w=w, h=h, minrangeset = 21, maxrangeset = 32, trans="rotate270.matrix", thermal.palette=rainbow1234pal)

Deconvolute temperature to raw and back to temperature

With thermal imaging analysis, there are at least 7 environmental parameters that must be known to convert raw to temperature. Sometimes, the parameters might have been incorrectly input by the user or changing the parameters is too cumbersome in the commercial software. *temp2raw()* is the inverse of raw2temp(), which allows you to convert an estimated temperature back to the raw values (i.e. deconvolute), using the initial object parameters used.

For example, convert a temperature estimated at 23 degrees C, under the default blackbody conditions:

temp2raw(23, E=1, OD=0, RTemp=20, ATemp=20, IRWTemp=20, IRT=1, RH=50, PR1=21106.77, PB=1501, PF=1, PO=-7340, PR2=0.012545258)

[1] 17994.06

Which yields a raw value of 17994.06 (using the calibration constants above). Now you can use *raw2temp* to calculate a better estimate of an object that has emissivity=0.95, distance=1m, window transmission=0.96, all temperatures=20C, 50 RH:

raw2temp(17994.06, E=0.95, OD=1, RTemp=20, ATemp=20, IRWTemp=20, IRT=0.96, RH=50, PR1=21106.77, PB=1501, PF=1, PO=-7340, PR2=0.012545258)

[1] 23.31223

Note: the default calibration constants for my FLIR camera will be used if you leave out the calibration data during this two step process, but it is more appropriate to look up your camera's calibrations constants using the flirsettings() function.

Export image or video

Finding a way to quantitatively analyse thermal images in R is a challenge due to limited interactions with the graphics environment. Thermimage has a function that allows you to write the image data to a file format that can be easily imported into *ImageJ*.

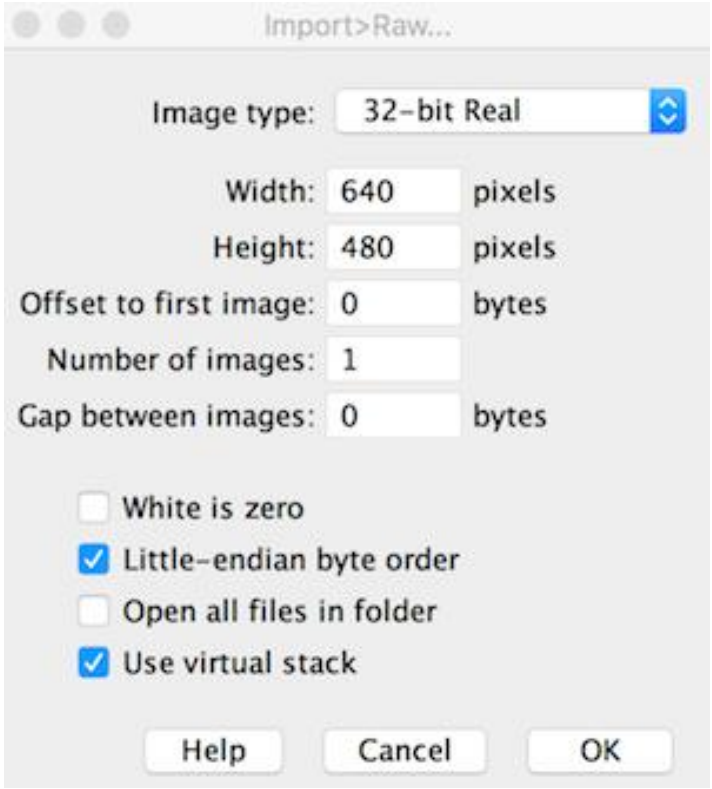
First, the image matrix needs to be transposed (t) to swap the row vs. column order in which the data are stored, then the temperatures need to be transformed to a vector, a requirement of the *writeBin* function. The function *writeFlirBin* is a wrapper for writeBin, and uses information on image width, height, frame number and image interval (the latter two are included for thermal video saves) but are kept for simplicity to construct a filename that incorporates image information required when importing to ImageJ:

writeFlirBin(as.vector(t(temperature)), templookup=NULL, w=w, h=h, I="", rootname="Uploads/FLIRjpg")

The raw file can be found here: https://github.com/gtatters/Thermimage/blob/master/Uploads/FLIRjpg_W640_H480_F1_I.raw?raw=true

Import raw file into ImageJ

The .raw file is simply the pixel data saved in raw format but with real 32-bit precision. This means that the temperature data (negative or positive values) are encoded in 4 byte chunks. *ImageJ* has a plethora of import functions, and the File–>Import–>Raw option provides great flexibility. Once opening the .raw file in ImageJ, set the width, height, number of images (i.e. frames or stacks), byte storage order (little endian), and hyperstack (if desired):



The image imports clearly just as it would in a thermal image program. Each pixel stores the calculated temperatures as provided from the raw2temp function above.

Convert FLIR JPG from R

If you have a lot of files and wish simply to analyse images in *ImageJ*, not in R, then you will want to bulk convert these files. The following methods are available in R, but are based on command line tools that are also described in https://github.com/gtatters/ThermimageBash/blob/master/README.md

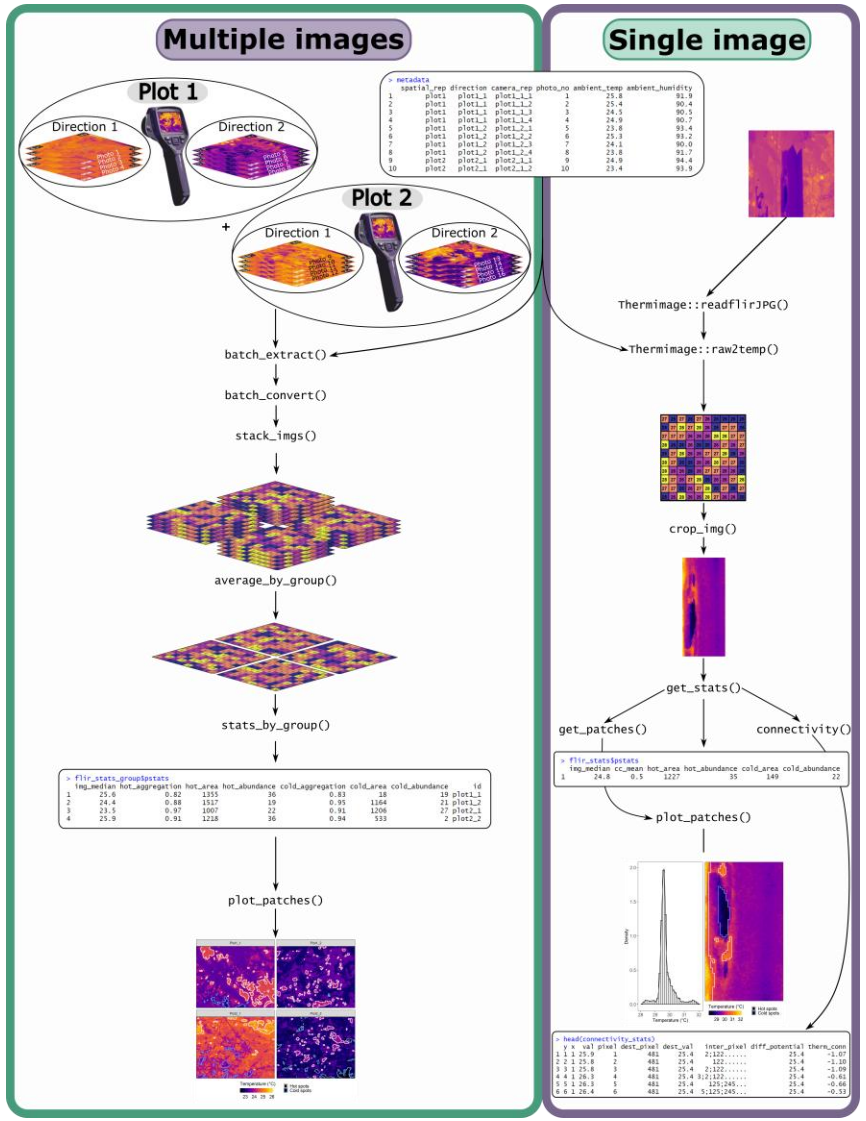
Bulk FLIR jpg file:

```
library(Thermimage)

exiftoolpath <- "installed"

f<-paste0("SampleFLIR/SampleFLIR.jpg")

# if JPG contains raw thermal image as TIFF, endian = "lsb"

# if JPG contains raw thermal image as PNG, endian = "msb"?
```

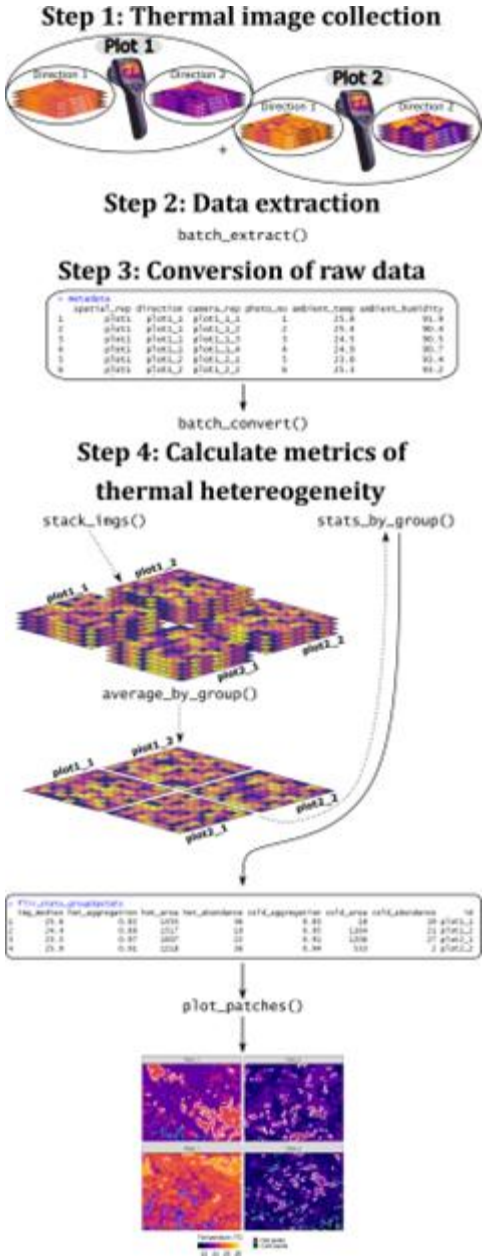
```
vals<-flirsettings(f)
w<-vals$Info$ImageWidth
h<-vals$Info$ImageHeight
res.in<-paste0(w,"x",h)

if(vals$Info$RawThermalImageType=="PNG") endian="msb"
if(vals$Info$RawThermalImageType=="TIFF") endian="lsb"

convertflirJPG(f, exiftoolpath="installed", res.in=res.in, endian=endian, outputfolder="",
verbose=FALSE)

cat("\n")
```

Converted files are in the output subfolder

ThermStats



Schematic summarising the key functions for processing groups of images (left) or a single image (right).



Overview of the main ThermStats functions and a typical workflow. Dashed lines indicate optional steps.

ThermStats is designed for biologists using thermography to quantify thermal heterogeneity. It uses the Thermimage package (Tattersall, 2019) to batch process data from FLIR thermal cameras, and takes inspiration from FRAGSTATS (McGarigal et al., 2012), SDMTools (VanDerWal et al., 2014), Faye et al. (2016) and Shi et al. (2016) to facilitate the calculation of various metrics of thermal heterogeneity for any gridded temperature data.

The package is available to download from GitHub using devtools:

devtools::install_github("rasenior/ThermStats")

library(ThermStats)

Converting raw data to temperature

Raw data are encoded in each thermal image as a 16 bit analog-to-digital signal, which represents the radiance received by the infrared sensor. The function *batch_convert* converts these raw data to temperature values using equations from infrared thermography, via a batch implementation of the function *raw2temp* in Thermimage. It uses the calibration constants extracted in batch_extract and environmental parameters defined by the user:

- Emissivity = the amount of radiation emitted by a particular object, for a given temperature.
- Object distance = the distance between the camera and the object of interest.
- Reflected apparent temperature = thermal radiation that originates from other objects and is reflected by the object of interest.
- Atmospheric temperature = the temperature of the atmosphere.
- Relative humidity = the relative humidity of the atmosphere.

```
# Define raw data
raw_dat <- flir_raw$raw_dat
# Define camera calibration constants dataframe
camera_params <- flir_raw$camera_params
# Define metadata
metadata <- flir_metadata
# Create vector denoting the position of each photo within metadata
photo_index <- match(names(raw_dat), metadata$photo_no)
# Batch convert
flir_converted <-   batch_convert(raw_dat = raw_dat, E = mean(c(0.982,0.99)),# Object distance =
# hypotenuse of right triangle where vertical side is 1.3 m (breast height) & angle down is 45°
OD = (sqrt(2))*1.3, # Apparent reflected temperature & atmospheric temperature =
 # atmospheric temperature measured in the field RTemp = metadata$atm_temp[photo_index],
  ATemp = metadata$atm_temp[photo_index], # Relative humidity = relative humidity measured in
#the field    RH = metadata$rel_humidity[photo_index], # Calibration constants from 'batch_extract'
PR1 = camera_params["PlanckR1"],  PB = camera_params["PlanckB"], PF =
camera_params["PlanckF"], PO = camera_params["PlanckO"], PR2 = camera_params["PlanckR2"],
 # Whether to write results or just return, write_results = FALSE)
```

Calculating thermal statistics

Statistics can be calculated for individual thermal images (in a matrix or raster format), or across multiple images within a specified grouping. The latter is useful for sampling designs where multiple images are collected at each sampling event to capture temperature across a wider sampling unit, such as a plot. In either case, statistics can include summary statistics specified by the user – for example, mean, minimum and maximum – as well as thermal connectivity (based on the climate connectivity measure of McGuire et al., 2016) and spatial statistics for hot and cold spots, identified using the G* variant of the Getis-Ord local statistic (Getis and Ord, 1996).

For an individual image, *get_stats* requires the user to specify the image and the desired statistics. Statistics can be calculated for geographic temperature data, in which case the user should also define the extent and projection of the data.

```
flir_stats <-    get_stats(         # The temperature dataset
        img = flir_converted$`8565`,
        # The ID of the dataset
        id = "8565",
        # Whether or not to calculate thermal connectivity
        calc_connectivity = FALSE,
        # Whether or not to identify hot and cold spots
        patches = TRUE,
        # The image projection (only relevant for geographic data)
        img_proj = NULL,
        # The image extent (only relevant for geographic data)
        img_extent = NULL,
        # The data to return
        return_vals = c("df", # Temperature data as dataframe
                        "patches", # Patch outlines
                        "pstats"), # Patch statistics dataframe
        # The summary statistics of interest
        sum_stats = c("median", "SHDI",
                      "perc_5", "perc_95"))
```

For grouped images, *stats_by_group* requires the user to supply a list of matrices or a raster stack, and (optionally) the metadata and the name of the variable in the metadata that defines the grouping. Table shows the metadata used in the code snippet, where photo number ('photo_no') defines individual temperature matrices, and the replicate identity ('rep_id') defines the grouping of photos. There are two replicates, 'T7P1' and 'T7P2', and each has two associated photos.

Table: Example metadata denoting the grouping ('rep_id') of different thermal images. Statistics can be calculated over multiple images within a group, using the function *stats_by_group*.

photo_no	rep_id	atm_temp	rel_humidity
8565	T7P1	24.00	96
8583	T7P1	24.00	96
8589	T7P2	23.25	98
8613	T7P2	23.50	96

By default, both *get_stats* and *stats_by_group* return a dataframe with patch statistics (Table below) for each image or group, respectively

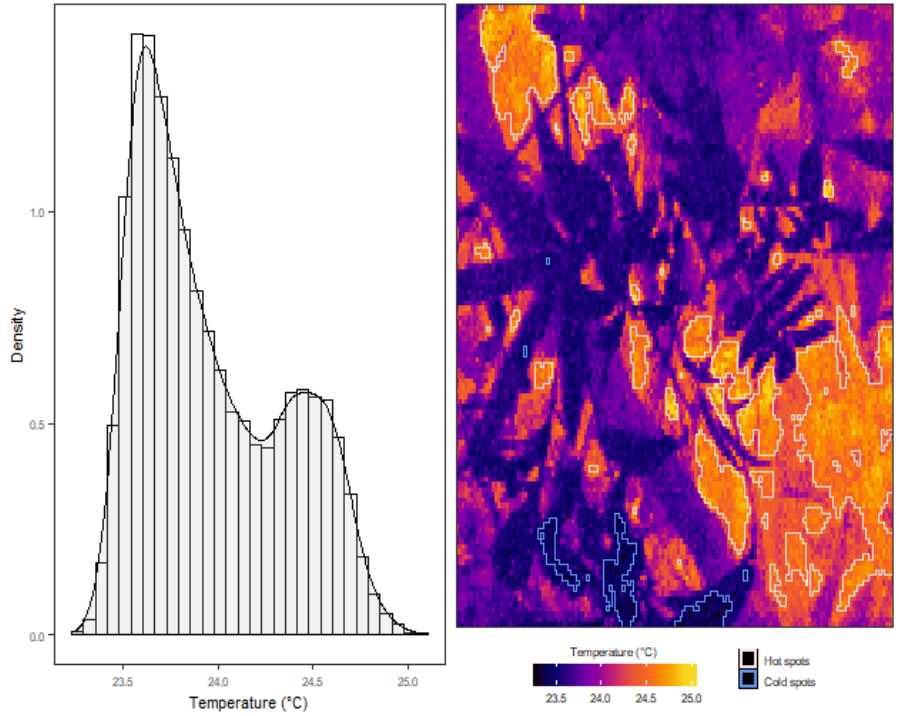
Table : A snippet of hot spot patch statistics returned by *stats_by_group*, which implements *get_stats* within groups.

Img_median	Img_perc_5	Img_perc_95	Img_SHDI	Hot_shape_index	Hot_aggregation
23.5	23	24.5	1.16	7.54	0.895
24.0	23	25.0	1.68	7.80	0.855

Plotting

In addition to patch statistics, *get_stats* can return (
1) the temperature dataset in a dataframe format, and
(2) a SpatialPolygonsDataFrame of its hot and cold spots.
The function *plot_patches* can then recreate the original thermal image overlaid with outlines of hot and cold spots, as well as the temperature distribution if *plot_distribution = TRUE* (Figure).

```
plot_patches(    # The raw temperature data
    df = flir_stats$df,
# The patch outlines
patches = flir_stats$patches)
```



The output of `plot_patches` includes a histogram and the original temperature data overlaid with outlines of hot and cold spots, identified using the G* variant of the Getis-Ord local statistic.

Step 1: thermal image collection

Surface temperature measurements at high spatial resolution can easily be collected in the field using a handheld thermal camera. We will use a FLIR model E40 camera, which costs ~US$..., weighs ... g, and takes ... measurements (... pixels) in a single photo (FLIR, 2016; Scheffers et al., 2017).

As with any field study, the sampling design should aim to achieve sufficient coverage over the study area and over time, such that the images are representative samples of the treatments of interest. For example a single image of the ground from 1 m away encompasses an area of 0.9 × 1.1 m (1 pixel ≅ 0.5 cm2) using a FLIR E40 camera (FLIR, 2016), and so it may be necessary to take multiple photos in different cardinal directions and at different times of day to effectively represent the temperature of a study plot (Scheffers et al., 2017; Senior, Hill, Benedick, & Edwards, 2018) (Figure 1).

There are various sources of the infrared radiation detected by a thermal camera, but we want to focus only on the radiation emitted by the object or scene of interest, which is a **function of its temperature**.

The amount of radiation emitted by a particular object, for a given temperature, depends on its **emissivity**. A perfect blackbody has an emissivity of 1, whereas surfaces that you are likely to photograph typically have an emissivity from 0.92 (for dry, bare soil; FLIR, 2016) to 0.99 (for green broadleaf forest; Snyder, Wan, Zhang, & Feng, 1998).

Additionally, the **temperature and relative humidity of the atmosphere and the distance between the object and the camera** will affect (a) the amount of emitted radiation that is absorbed by the atmosphere and (b) the amount of background radiation, with some of this also being reflected by the object (reflected apparent temperature).

To accurately quantify surface temperature, environmental parameters (emissivity, reflected apparent temperature, atmospheric temperature, atmospheric relative humidity and object distance) can be s**et in the camera or defined during data processing** (see 'Step 3: Conversion of raw data').

In the latter approach the user can measure atmospheric temperature and relative humidity concurrently with thermal image collection, and set these parameters during image processing. Object distance should be minimized and kept constant where possible.

Emissivity can either be estimated from the literature (cf. Scheffers et al., 2017) or sampled in the field (FLIR, 2016). Temperature measurements will be more accurate if there is **less variation in surface emissivity** within each image. That said, common components of the ground surface, such as leaves and soil, have very similar, high emissivity (> 0.9; Snyder et al., 1998) and thus the impact of emissivity variation on temperature measurements is relatively low. Likewise, reflected apparent temperature can be sampled in the field (FLIR, 2016), but for **high emissivities and short object distances, relatively little radiation is reflected and thus apparent temperature can be assumed to equal the atmospheric temperature** (Tattersall, 2017).

Step 2: data extraction

Images are extracted in batch by the function *batch_extract*, which requires only the path to a directory of FLIR images, and the external software *ExifTool*. Instructions for installing ExifTool can be found here: https://exiftool.org/ .

Step 3: conversion of raw data

The raw values embedded in a FLIR thermal image can be converted to temperature in °C using equations from infrared thermography (FLIR, 2016; Tattersall, 2017) in the function *batch_convert*. Temperature conversion will be more accurate when the user defines the environmental parameters, described in Step 1. Notably, default emissivity is 1, but should usually take a value between 0.95 and 0.97 (Tattersall, 2017), and relative humidity is highly dependent on environment and weather conditions. Conversion of raw data also requires various calibration constants that are specific to each camera; these are retrieved automatically in *batch_extract* to be passed directly to *batch_convert*.

Step 4: calculate metrics of thermal hetereogeneity

1: Pixel level statistics

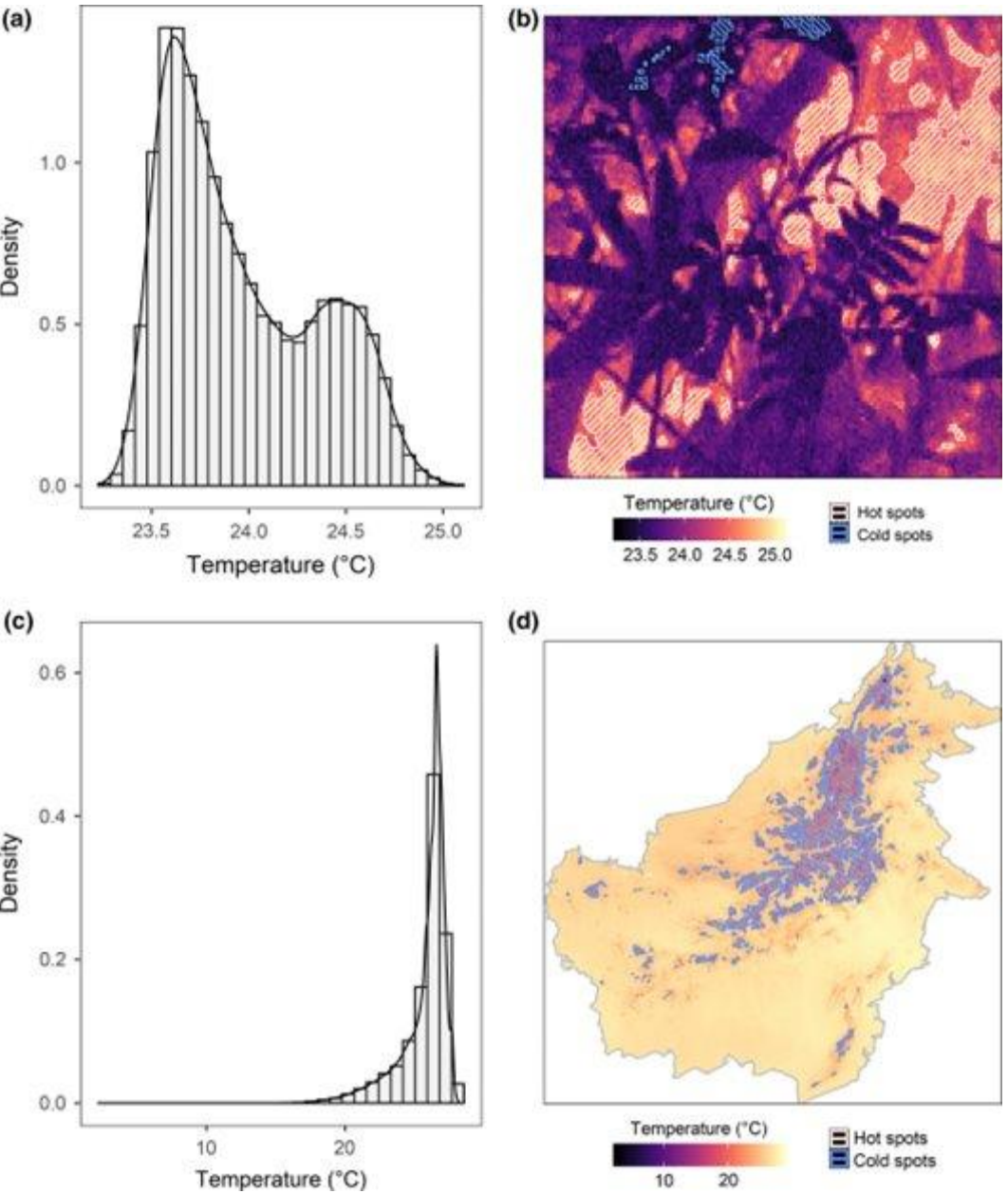
The most relevant metrics to quantify thermal heterogeneity depend on the particular research questions. The function *get_stats* takes a single thermal dataset, in the form of a matrix or raster (Hijmans, 2018), and calculates user-defined summary statistics across all pixels. Standard summary statistics could include measures such as mean and standard deviation, but may also include metrics such as thermal richness (the number of unique temperature values) and thermal diversity indices (cf. Faye et al., 2017). Several helper functions are available to implement less standard summary statistics. On the basis of discussions in Faye et al. (2017) and Shi et al. (2016), Table 1 provides some recommended statistics.

Summary statistic	Description
Average temperature	Provides context for all other statistics, and could be used as a measure of the macroclimate for small objects. The median is more robust than the mean to spurious extreme values that can sometimes arise in thermal images.
Temperature extremes	While more rarely encountered, extreme values can be more significant for some objects. The difference between extremes provides a measure of thermal diversity/stability (Shi et al., 2016), whereas the difference between extremes and average temperature provides a measure of the potential for thermal buffering. Again, we suggest the 5th and 95th percentiles are more robust to spurious extreme values than the minimum and maximum (respectively).
Temperature variability	Over space and time, the standard deviation or coefficient of variation in temperature represents another measure of thermal stability (Shi et al., 2016).
Thermal diversity indices	Capture both the richness and evenness of different temperatures. As discussed by Faye et al. (2016), Shannon's thermal diversity index quantifies how reliably one can predict the temperature of a pixel sampled at random from the temperature data. Simpson's thermal diversity index is similar; it captures the likelihood of two pixels being the same temperature (or temperature class) when sampled at random.
Thermal connectivity	Calculated for each pixel as the potential for temperature change, which is the maximum temperature difference that can be achieved by traversing a gradient of hotter to cooler pixels. Adapted from McGuire et al. (2016).

2: Patch-level statistic

The function *get_stats* identifies hot and cold spots in thermal images using a standalone function *get_patches*. Hot and cold spots are based on the G* variant of the Getis-Ord local statistic (Getis & Ord, 1996), calculated using the spdep package (Bivand & Piras, 2015). The statistic is calculated for individual pixels by comparing the local weighted average of the pixel and its neighbours to the global average. High positive values exceeding the Z-value threshold (defined according to the sample size; Getis & Ord, 1996) are classified as hot spots, and low negative values as cold spots. Several spatial statistics are then calculated to characterize the hot and cold spots (Table 2; cf. Faye et al., 2016). There is an option to return patch outlines as a SpatialPolygonsDataFrame, which can be plotted on the temperature data using *plot_patches* alongside an (optional) histogram of the temperature distribution (Figure 2).

Patch statistic	Description
Area (absolute)	Total number of pixels
Area (proportion)	Proportion of all pixels which are inside hot/cold spots
Abundance	Number of distinct hot/cold spots
Density	Number of hot/cold spots per unit area
Shape index	Irregularity in shape of hot/cold spots, with 1 being perfectly regular (i.e. a square; McGarigal et al., 2012)
Aggregation index	Degree of clustering in space of hot/cold spots, with zero representing no clustering (He, DeZonia, & Mladenoff, 2000)
Patch cohesion index	Physical connectedness of hot/cold spots (Schumaker, 1996)



Examples of temperature frequency distribution (left column) and spatial distribution (right column) for temperature data from a thermal image (top row) and from WorldClim2 (bottom row; Fick & Hijmans, 2017). Pixels are shaded from cold (black) to hot (beige). Clusters of extreme values were identified by calculating the G* variant of the Getis-Ord local statistic for each pixel (Getis & Ord, 1996), with high positive values assigned to hot spots (outlined and hatched in pink) and low negative values to cold spots (outlined and hatched in blue)

3: multiple images

We assume that for most users the spatial unit of replication will comprise multiple thermal images. In this case, the user can specify a grouping variable in *stats_by_group*. Matrices from each group will be bound together and *get_stats* applied over the combined matrix. Processing times increase with more or larger images, although this can be managed by disabling the calculation of hot and cold spots or of thermal connectivity (patches = FALSE and connectivity = FALSE), since these are the most computationally intensive statistics. We assume that images are not adjacent in space, and therefore pad matrices with NA values before binding. Table 3 gives an example of the output from stats_by_group. Users can also average across repeated samples of the same area using *average_by_group*, the output of which can be passed directly to *stats_by_group*. Raster stacks are generally quicker to process, and can be created from a list of matrices using *stack_imgs*.

photo_no	rep_id	atm_temp	rel_humidity
8565	T7P1	24.00	96
8583	T7P1	24.00	96
8589	T7P2	23.25	98
8613	T7P2	23.50	96

Example applications

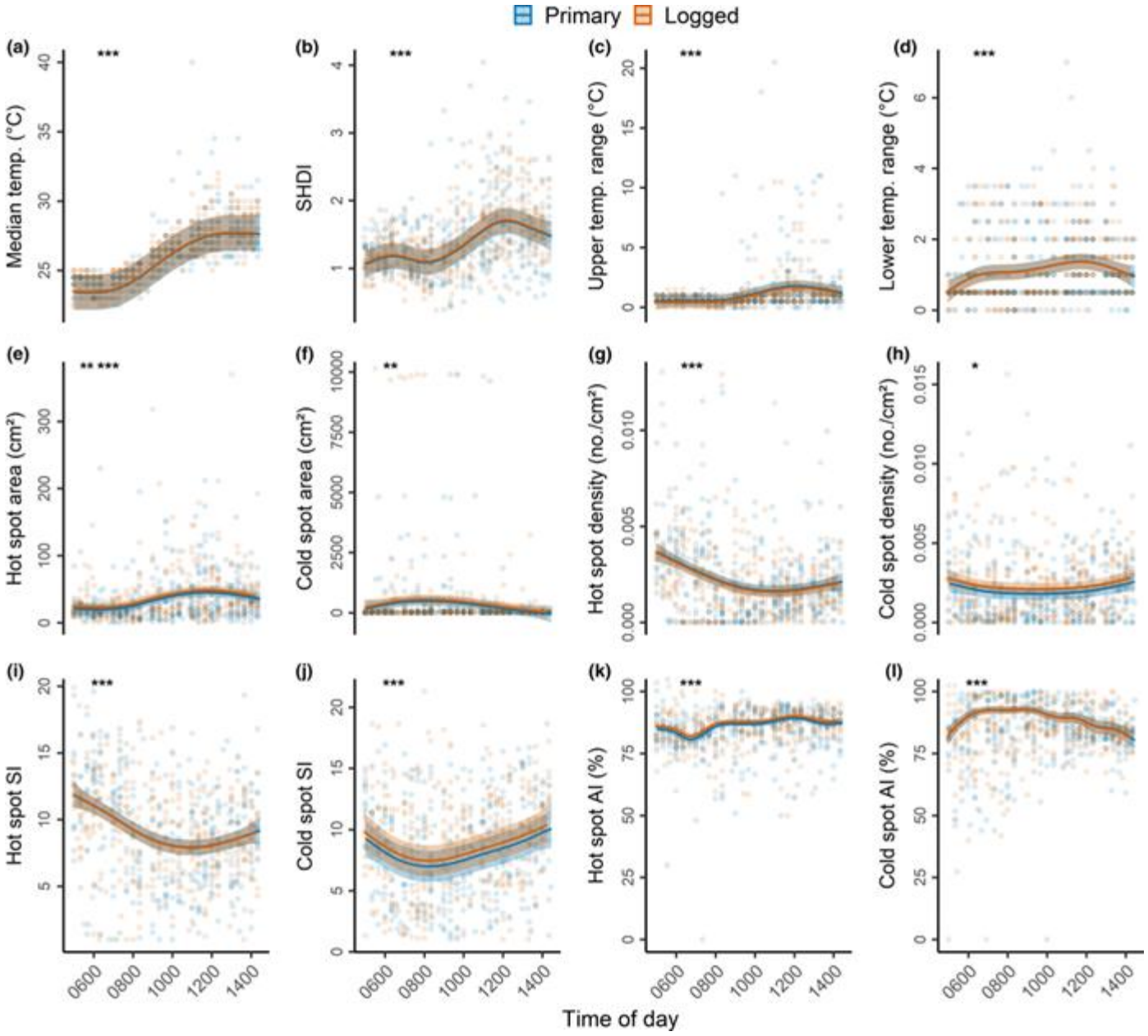
Time variation of thermal heterogeneity

Samples of surface temperature in a large area of contiguous forest in Malaysian Borneo in the years 2014 and 2015

In both years, photos were taken at points spaced every 125 m along existing transects, with six transects in undisturbed primary forest (Danum Valley Conservation Area; 4°57045.2″N, 117°48010.4″E), and six transects in adjacent forest that had been commercially selectively logged twice between 1987 and 2007 (Ulu Segama-Malua Forest Reserve, 4°57042.8″N, 117°56051.7″E). Points were sampled repeatedly from the coolest to the hottest part of the day (05:00–14:30 hr). In each sampling event, thermal images were taken in four orthogonal directions, with the camera held at breast height and pointing 45° downwards (relative to the ground). In total we collected 2,972 photos across 144 points. For full details see Scheffers et al. (2017) and Senior et al. (2018).

Data were extracted using *batch_extract* and converted using *batch_convert*. We focused on the following summary statistics from Table 1: median temperature; Shannon Diversity Index; upper temperature range (95th percentile—median); and lower temperature range (median—5th percentile). For hot and cold spots separately, we calculated the following spatial statistics (Table 2): average area per patch (total area divided by number of patches, to correct for points with missing photos); average number of patches per unit area (density); Shape Index and Aggregation Index. We used *stats_by_group* to calculate each metric across all four photos taken each time a point was sampled (one photo in each direction).

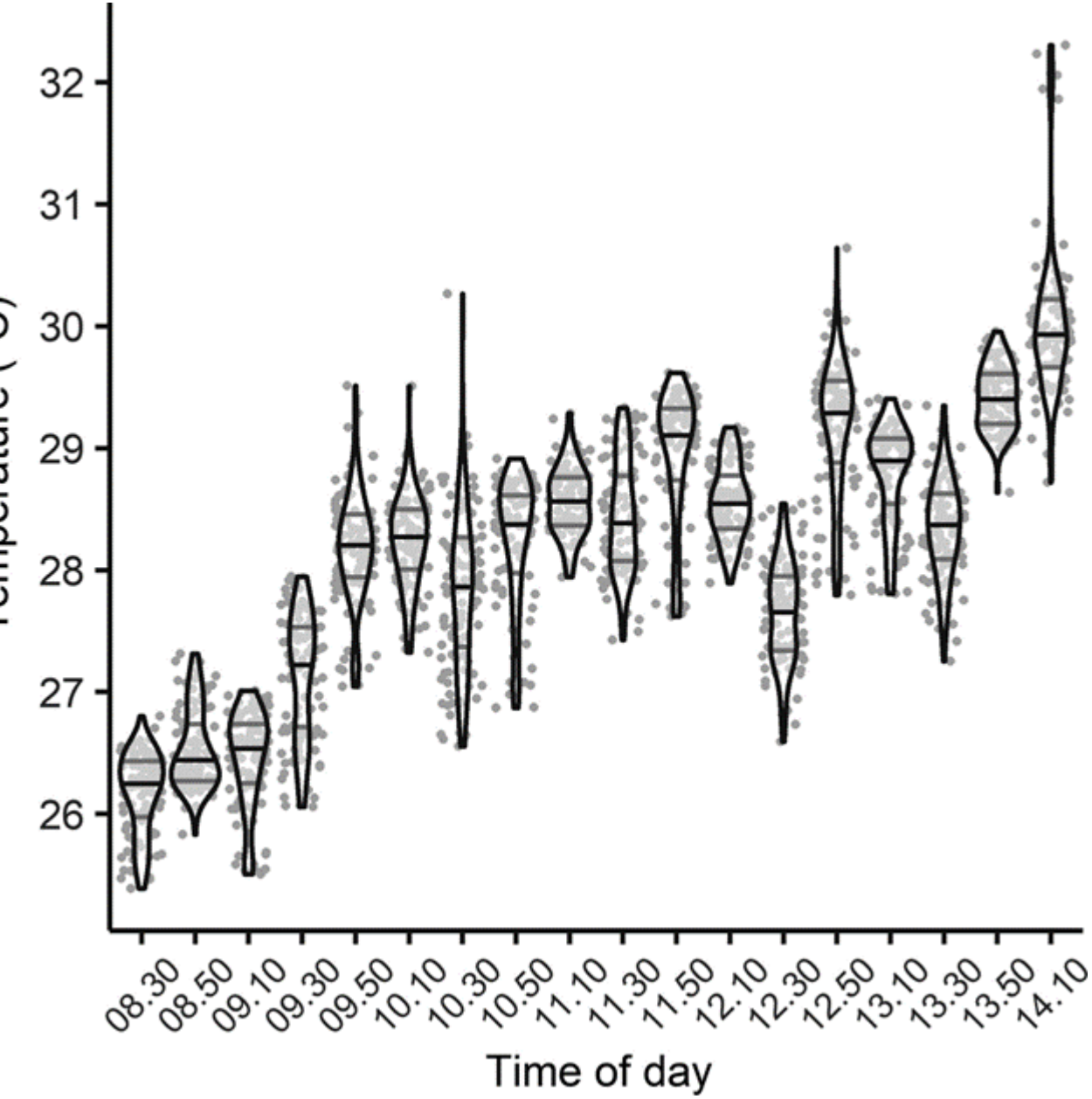
To model the various thermal heterogeneity metrics against time of day and forest type (categorical: primary or logged) we fit Generalized Additive Mixed Effects Models (GAMMs), using the gamm4 package (Wood & Scheipl, 2017) in R. All measures of thermal heterogeneity were comparable between primary and unlogged forest ($p > .05$), but varied from the coolest time of day (~06:00 hr) to the hottest (~12:30 hr). Of particular note is the peak in thermal diversity (Shannon Diversity Index) at noon, and the different temporal patterns of hot spots (median temperature: 27°C) compared to cold spots (median temperature: 25°C).



Trends in various measures of thermal heterogeneity over the day (06:00–14:30 hr) for fine-scale temperature data collected using a thermal camera in primary (blue) and logged forests (orange). From left to right and top to bottom, the metrics are: median temperature (a); thermal Shannon Diversity Index (SHDI) (b); 95th percentile minus median temperature (c); median temperature minus 5th percentile (d); the average area (cm2) per hot spot (e); the average area (cm2) per cold spot (f); the number of hot spots per unit area (g); the number of cold spots per unit area (h); the Shape Index (SI) of hot spots (i); the Shape Index (SI) of cold spots (j); the Aggregation Index (AI) of hot spots (k); and the Aggregation Index (AI) of cold spots (l). Solid lines are model-predicted values with 95% confidence intervals. Semitransparent background points represent the raw data. Statistically significant differences are indicated by asterisks: .01 < p < .05 (*); .001 < p < .01 (**) and p < .0001 (***)

Averaging over repeated images

Averaging across repeated images of the same area could help to reduce unwanted variation, such as that introduced by the camera or photographer. In this example, thermal images were collected at nine points ('spatial replicate') spaced along existing transects in undisturbed primary forest (Danum Valley Conservation Area; 4°57045.2″N, 117°48010.4″E). Each point was visited at two different times ('temporal replicate') on the same day. On each visit to a point, between three and five photographs were taken sequentially ('camera replicate') between three and five times, with the photographer moving away each time and resetting their position ('photographer replicate'). To smooth variation largely introduced by the camera and photographer, we averaged within temporal replicates using the function *average_by_group*. The function *plot_stack* is ideal for visualizing these data (Figure 4), as well as functions in the package rasterVis (Perpiñán & Hijmans, 2018).

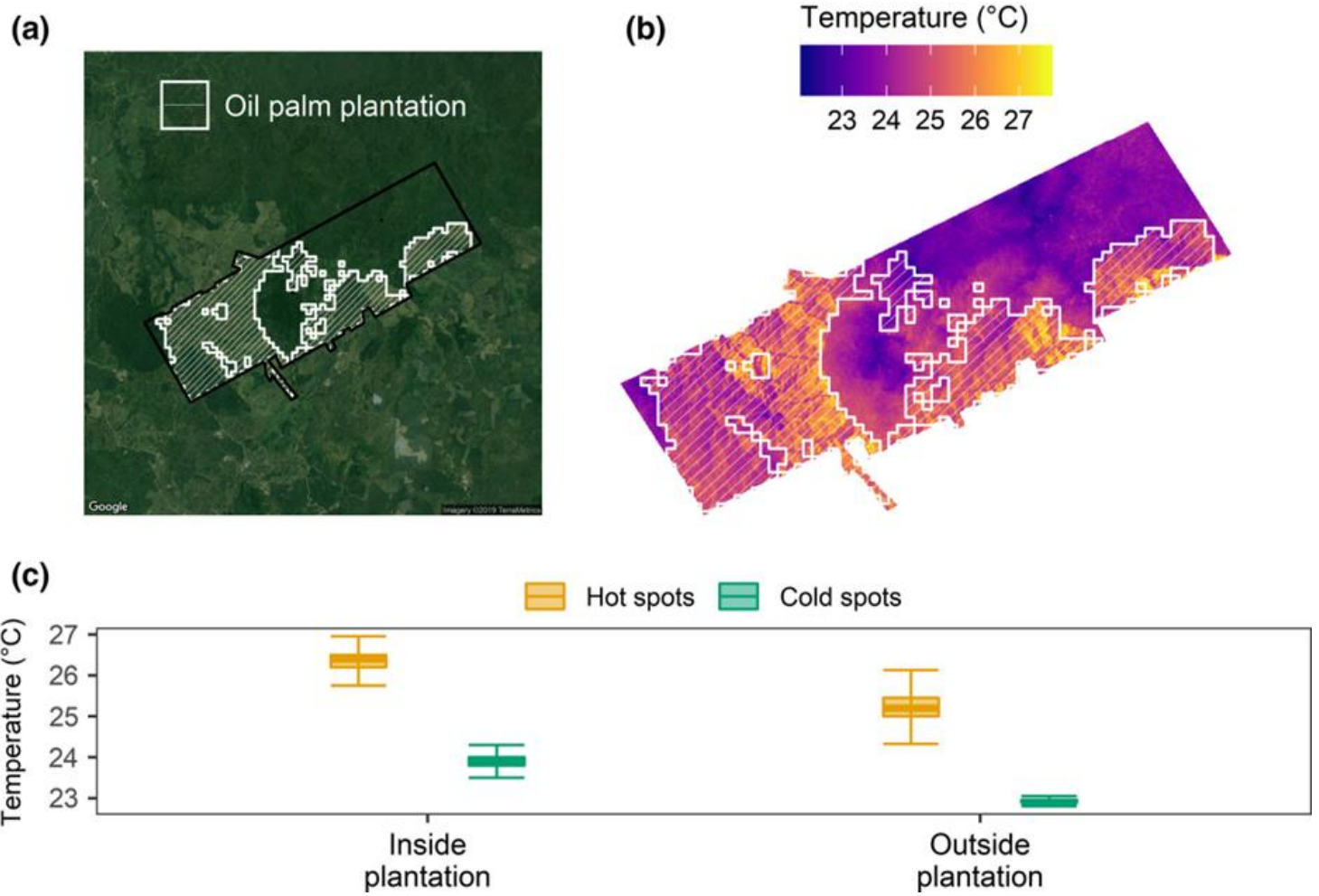


Violin plots of temperature distribution across the raster of each distinct replicate in time. Each datapoint represents one pixel of the raster, with 100 pixels sampled at random. Each raster is the average of multiple thermal images collected at the same time and location, averaged within temporal replicates using the function average_by_group

Violin plots are used when you want to observe the distribution of numeric data, and are especially useful when you want to make a comparison of distributions between multiple groups. The peaks, valleys, and tails of each group's density curve can be compared to see where groups are similar or different.

Other gridded temperature data

Analytical functions in ThermStats are designed for any gridded temperature data. We illustrate this using a mean annual temperature layer (50 × 50 m resolution) estimated from canopy structure and topographic variables measured using LIDAR (Jucker et al., 2018). The layer encompasses forest and oil palm plantation within the Stability of Altered Forest Ecosystems (SAFE) project (Ewers, Rebaudo, Yánez-Cajo, Cauvy-Fraunié, & Dangles, 2011)". We created a raster stack with one layer for cells inside the plantation boundary and one for cells outside, and used stats_by_group to identify hot and cold spots within each layer. The patch characteristics calculated by stats_by_group include upper and lower quantiles and the median, which together characterize the temperature distribution and can be used to construct boxplots (Figure 5). Qualitatively, we demonstrate that both hot and cold spots were hotter inside the oil palm plantation than the neighbouring forest.



Panel (a) shows the study area in the Stability of Altered Forest Ecosystems (SAFE) project, overlaid with the boundary of oil palm plantations (Ewers et al., ; Kahle & Wickham,). Variation in modelled mean annual temperature is shown in panel (b) (Jucker et al.,). Panel (c) shows the temperature distribution of hot spots (orange) and cold spots (green), inside and outside plantations

Python libraries

- flirpy (https://pypi.org/project/flirpy/)
- Flir Image Extractor CLI (https://pypi.org/project/flirimageextractor/)
- FLIR thermal tools (https://github.com/susanmeerdink/FLIR_thermal_tools)
- ...

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Erika Brattich

Department of Physics and Astronomy «Augusto Righi», University of Bologna

Erika.brattich@unibo.it